INSTITUTO FEDERAL
São Paulo

Orientação a objetos em PHP

PROF. DR. EDER PANSANI
epansani@gmail.com / epansani@ifsp.edu.br

Conteúdo da aula

INSTITUTO FEDERAL
São Paulo

Nesta aula veremos alguns do conceitos básicos e tópicos importantes sobre a **Orientação e Objetos no PHP**.

PROF. DR. EDER PANSANI
EPANSANI@GMAIL.COM / EPANSANI@IFSP.EDU.BR

INSTITUTO FEDERAL
São Paulo

Introdução

Introdução

A orientação a objetos é um **paradigma** que representa uma **filosofia para construção de sistemas**.

Introdução

INSTITUTO FEDERAL
São Paulo

Ao invés de construir sistemas formados por um **conjunto de procedimentos e variáveis**, nem sempre agrupadas, na orientação a **objetos** utiliza-se uma **ótica mais próxima do mundo real**.

Introdução

São criadas estruturas que carregam dados e possuem comportamentos próprios, além de trocarem mensagens entre si com objetivo de formar algo maior, um sistema.

Introdução

A **abordagem orientada a objetos é o paradigma de programação mais utilizado atualmente**, porém nem sempre foi assim.

Introdução

INSTITUTO FEDERAL
São Paulo

No caso específico da linguagem **PHP**, ela **não foi criada para ser orientada a objetos**. O suporte a OO começou na versão 3, foi melhorada na versão 4 e foi plenamente suportada a partir da versão 5.3 (2009).

Introdução

INSTITUTO FEDERAL
São Paulo

Antes do paradigma OO os desenvolvedores PHP utilizam uma abordagem comumente chamada de "**orientação a funções**", que consiste em organizar as funções mais utilizadas em arquivos específicos e incluí-los usando ***include*** ou ***require***.

Introdução

No entanto, essa abordagem era essencialmente uma abordagem **procedural** (uso do paradigma de programação procedural).

INSTITUTO FEDERAL
São Paulo

Programação procedural

Programação procedural

A programação procedural **foi** a abordagem mais utilizada pelos programadores PHP durante muito tempo.

Programação procedural

Ela consiste basicamente na construção de procedimentos (funções ou páginas isoladas) que resolvem determinado problema.

Programação procedural

Na programação procedural **é possível ter um reaproveitamento de código**, porém devido a sua natureza, o **reuso de procedimentos torna-se mais complicado**.

Programação procedural

Na **programação procedural** ocorre naturalmente uma **fragmentação de códigos**, tornando a **organização e reuso mais dependentes da habilidade dos desenvolvedores**.

Programação procedural

O **PHP** é uma **linguagem *multiparadigma***, ou seja, suporta mais de um paradigma de programação. Isso significa que é possível utilizá-lo para programar de forma procedural ou orientada a objetos.

Programação procedural

O exemplo a seguir mostra como seria uma página composta por funções que possuem finalidades específicas. É importante frisar que este não é um exemplo funcional.

Programação procedural

INSTITUTO FEDERAL
São Paulo

Exemplo de programação procedural

```php
<?php

function total_debitos_cliente($id_cliente){  }
function registra_venda($id_cliente, $data){  }
function registra_venda_item($id_venda, $item){  }
function consulta_estoque($produto){  }
function consulta_valor($produto){  }
function gera_financeiro($id_cliente, $total, $parcelas){  }

function conclui_venda($id_cliente, $parcelas, $itens){
    if(total_debitos_cliente($id_cliente) > 0){
        return false;
    }
    $id_venda = registra_venda($id_cliente, date('Y-m-d'));
    if($itens){
        foreach ($itens as $item) {
            if(consulta_estoque($item) <= 0){
                return false;
            }
            $total += consulta_valor($item);
            registra_venda_item($id_venda, $item);
        }
    }
    gera_financeiro($id_cliente, $total, $parcelas);
}

conclui_venda(5, 5, array(1,2,3));
```

```php
<?php

function total_debitos_cliente($id_cliente){  }
function registra_venda($id_cliente, $data){  }
function registra_venda_item($id_venda, $item){  }
function consulta_estoque($produto){  }
function consulta_valor($produto){  }
function gera_financeiro($id_cliente, $total, $parcelas){  }

function conclui_venda($id_cliente, $parcelas, $itens){
    if(total_debitos_cliente($id_cliente) > 0){
        return false;
    }
    $id_venda = registra_venda($id_cliente, date('Y-m-d'));
    if($itens){
        foreach ($itens as $item) {
            if(consulta_estoque($item) <= 0){
                return false;
            }
            $total += consulta_valor($item);
            registra_venda_item($id_venda, $item);
        }
    }
    gera_financeiro($id_cliente, $total, $parcelas);
}

conclui_venda(5, 5, array(1,2,3));
```

TUTO
ERAL
Paulo

Programação procedural

A partir deste trecho de código pode-se discutir algumas **conclusões** sobre este tipo de programa:

Programação procedural

INSTITUTO FEDERAL
São Paulo

Um procedimento não retém estado

Cada procedimento (*function*) recebe parâmetros de entrada, os processa e devolve uma resposta, porém não armazena informações pra uso futuro.

Programação procedural

INSTITUTO FEDERAL
São Paulo

Não implica em organização modular

O paradigma procedural não leva o desenvolvedor a organizar o programa de maneira modular, agrupando funcionalidades em comum.

Programação procedural

Por exemplo, funcionalidades relativas as vendas e ao controle de estoque são distintas, logo deveriam estar separadas. Neste paradigma a organização depende muito mais de uma organização do desenvolvedor.

Programação procedural

INSTITUTO FEDERAL
São Paulo

Não leva a uma organização de nomes

No paradigma procedural a organização da nomenclatura de funções e procedimentos fica a cargo do desenvolvedor, permitindo assim uma “liberdade” que pode levar a desorganização.

Programação procedural

INSTITUTO FEDERAL
São Paulo

Não facilita o reuso

Como o paradigma não induz o desenvolvedor a organizar funcionalidades de maneira modular, tampouco a padronização de nomenclatura será naturalmente mais...

Programação procedural

...**difícil** localizar procedimentos já construídos e reaproveitá-los. Poderão então ocorrer situações em que um mesmo código será reescrito pois o desenvolvedor não localizou uma implementação prévia.

Programação procedural

INSTITUTO FEDERAL
São Paulo

Mais suscetível a mudanças

Em razão das dificuldades para localizar funcionalidades, bem como suas dependências um programa procedural é mais suscetível a mudanças.

Programação procedural

Por exemplo, ao alterar uma função pode ser complicado identificar com precisão outros blocos que fazem uso da mesma.

Programação procedural

Além é claro de que a fragmentação de códigos para funcionalidades iguais refletir em retrabalho para implementação de possível mudanças em regras de negócio.

INSTITUTO FEDERAL
São Paulo

Programação Orientada a objetos (POO)

Orientação a objetos

INSTITUTO FEDERAL
São Paulo

O paradigma de **programação orientado a objetos (POO)** envolve desde a modelagem do sistema até sua programação por meio do **uso de objetos e relacionamento entre estes objetos**.

Orientação a objetos

INSTITUTO FEDERAL
São Paulo

Um **objeto** é uma estrutura de programação que representa uma **entidade**.

Mas o que é exatamente uma entidade?

Orientação a objetos

INSTITUTO FEDERAL
São Paulo

Uma **entidade** pode ser compreendida como **uma representação de algo que existe no mundo real** e que tem algum significado para o software, como uma pessoa, uma venda ou um fornecedor.

Orientação a objetos

INSTITUTO FEDERAL
São Paulo

No entanto, um **objeto** também pode ser algo necessário apenas por questões técnicas, como um **formulário**, um **relatório**, uma **conexão com o banco** ou a representação de um ***web service***.

Orientação a objetos

INSTITUTO FEDERAL
São Paulo

Independentemente do contexto, **uma aplicação pode ser caracterizada como orientada a objetos quando é formada exclusivamente por um conjunto de objetos que trocam mensagens entre si para resolução de um programa complexo**.

Orientação a objetos

INSTITUTO FEDERAL
São Paulo

Um programa orientado a objetos funciona baseado na **comunicação** por meio de **mensagens** que alteram os **atributos** de cada objeto. Estes **atributos** são as **características** de **cada objeto** distinto.

Orientação a objetos

INSTITUTO FEDERAL
São Paulo

Na orientação a objetos **cada objeto tem uma responsabilidade única**. Isso significa que um objeto deve além de administrar seus atributos, ter **um conjunto de funcionalidades coerentes**.

Orientação a objetos

INSTITUTO FEDERAL
São Paulo

Desta forma é natural que a orientação a objetos pode facilitar a manutenção, pois, **características e funcionalidades sobre uma entidade estão agrupadas em um determinado objeto** e não espalhadas pelo sistema.

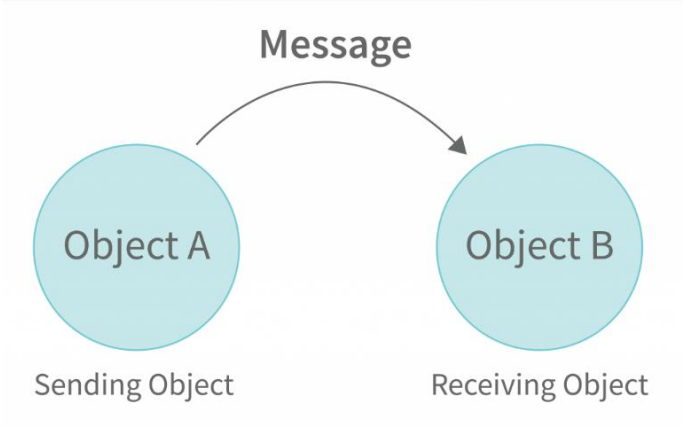
Orientação a objetos

INSTITUTO FEDERAL
São Paulo

Para construir um **sistema completo de maneira orientada a objetos**, é preciso construir objetos que tratam diferentes aspectos de uma aplicação como **modelo de negócios, interface, persistência**, entre outros.

Orientação a objetos

É preciso também fazer com que estes **objetos com diferentes responsabilidades** **“conversem”** **entre si** para realização das atividades necessárias.



Orientação a objetos

INSTITUTO FEDERAL
São Paulo

Vários objetos podem ser criados durante a execução de uma aplicação, porém antes de criar um objeto é necessário declarar formalmente sua estrutura

Orientação a objetos

INSTITUTO FEDERAL
São Paulo

Essa estrutura chama-se **classe** e é nela onde são declarados os atributos (características) e métodos que implementarão seu comportamento.

INSTITUTO FEDERAL
São Paulo

Classes

Classes

INSTITUTO FEDERAL
São Paulo

Em PHP a definição de uma classe começa com a palavra-chave ***class***, seguida do nome da classe, um par de colchetes que englobam as definições de propriedades e métodos pertencentes à classe.

Classes

INSTITUTO FEDERAL
São Paulo

Sintaxe

```php
<?php
class Fruit {
  // code goes here...
}
?>
```

Classes

Ao declarar os atributos de uma classe é necessário especificar qual o **encapsulamento (ou a visibilidade)** de cada um deles, que pode ser ***public, protected or private***.

INSTITUTO FEDERAL
São Paulo

Encapsulamento

Encapsulamento

O encapsulamento é um recurso interessante da orientação a objetos, pois ele provê um mecanismo de **proteção de acesso às características de uma determinada classe**.

Encapsulamento

INSTITUTO FEDERAL
São Paulo

Isso é importante pois podem existir certas propriedades que **não devem ser manipuladas fora de sua classe** ou ainda devam passar por **validações** para **garantia das regras de negócio**.

Encapsulamento

As três opções de encapsulamento ou visibilidade são ***public***, ***private*** ou ***protected*** cada uma prove diferentes formas de acesso.

Encapsulamento

Public, estes atributos podem ser acessados livremente na própria classe ou em classes dependentes a partir de programas que instanciam esta classe. É a visibilidade mais permissiva.

Encapsulamento

Private, de forma resumida é o contrário do public. Atributos *private* podem ser acessados apenas na própria classe, não sendo permitido sua manipulação direta e sim por métodos *"get" "set"*.

Encapsulamento

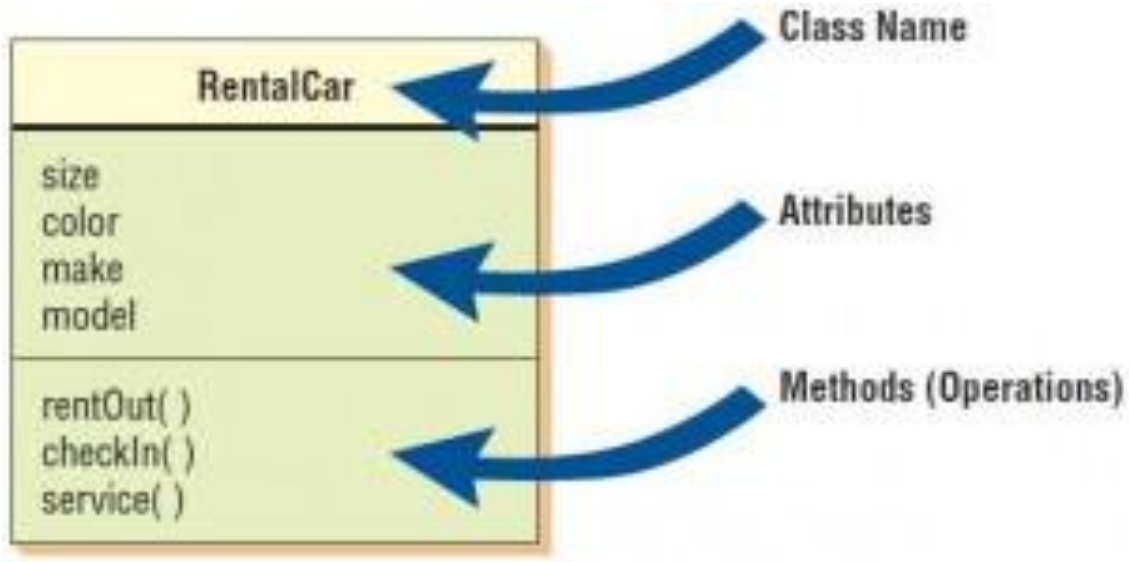
Protected, atributos deste tipo podem ser manipulados em sua própria classe e em classes descendentes, porém não podem ser manipulados diretamente em programas que fazem instâncias desta classe, sendo necessário os "*gets*" e "*sets*"

INSTITUTO FEDERAL
São Paulo

Atributos

Atributos

São os elementos que definem a estrutura da classe, suas **características**, particularidades que a definem.



Atributos

INSTITUTO FEDERAL
São Paulo

Os **atributos são como as variáveis** de uma determinada classe.

Podem ser de dois tipos básicos, de instância e de classe (também conhecidos como estáticos).

Atributos

É recomendável que se apliquem políticas de **encapsulamento aos atributos**, e que estes sejam manipulados apenas pelos métodos adequados.

Métodos

Métodos

Métodos são essencialmente a forma de um objeto possibilitar a manipulação e definir o comportamento permitido ao mundo externo, ou mais especificamente aos programas que instanciam esses objetos.

Métodos

Tais requisitos são obtidos com o uso dos métodos.

Um método é uma função que faz parte de uma classe e representa um comportamento que ela precisa expor ao ambiente externo.

Métodos

INSTITUTO FEDERAL
São Paulo

Por exemplo, em uma **classe Produto** poderíamos ter métodos ***aumentarEstoque()***, ***diminuirEstoque()***, ***reajustarPreco()*** que seriam responsáveis por manipular os atributos protegidos no interior da classe.

Métodos

Eles são **essenciais** pois é desejável que os atributos da classe produto seja privados (para que não sejam acessados diretamente) permitindo que somente os métodos padronizados, que façam todas as etapas de manipulação de estoque alterem os valores de um **[objeto] Produto**.

Métodos

INSTITUTO FEDERAL
São Paulo

Os métodos podem ou não receber atributos como **parâmetros** e também podem ter sua visibilidade configurada assim como os atributos.

Classes

INSTITUTO FEDERAL
São Paulo

Voltando às classes, observe a definição de uma classe com atributos (propriedades) e métodos:

```php
<?php
class Fruit {
  // Properties
  public $name;
  public $color;

  // Methods
  function set_name($name) {
    $this->name = $name;
  }
  function get_name() {
    return $this->name;
  }
}
?>
```

```php
<?php
class Fruit {
  // Properties
  public $name;
  public $color;

  // Methods
  function set_name($name) {
    $this->name = $name;
  }
  function get_name() {
    return $this->name;
  }
}
?>
```

Classes

Em determinadas situações, métodos e atributos podem pertencer a uma classe e não a um objeto específico. Eles são conhecidos como **estáticos**.

Classes

A diferença crucial é que **métodos ou atributos estáticos são comuns a suas classes**, isto é, são **compartilhados entre todos os objetos** de uma mesma classe.

Classes

INSTITUTO FEDERAL
São Paulo

Exemplo,

Syntax

```php
<?php
class ClassName {
  public static function staticMethod() {
    echo "Hello World!";
  }
}
?>
```

Classes

INSTITUTO FEDERAL
São Paulo

Para acessar um método estático pode-se utilizar o nome da classe seguido por dois sinais de dois pontos (::)

Sintaxe

```
ClassName::staticMethod();
```

Classes

As propriedades estáticas segue basicamente o mesmo princípio, porém não as abordaremos em detalhes.

INSTITUTO FEDERAL
São Paulo

Recursos Avançados

Recursos Avançados

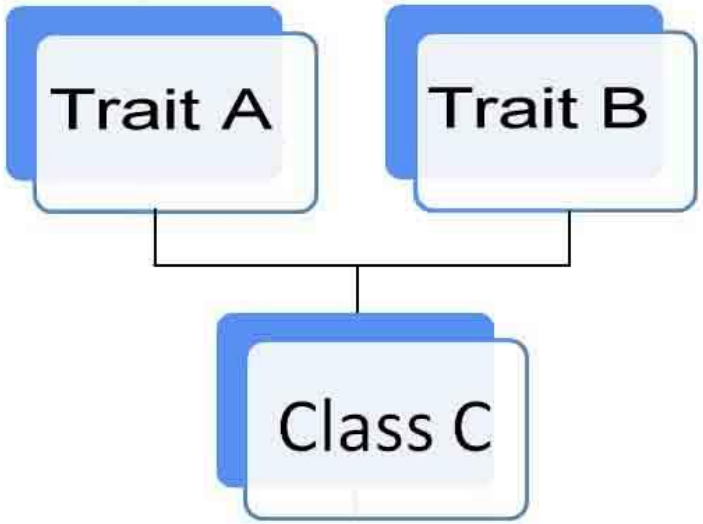
INSTITUTO FEDERAL
São Paulo

Há ainda alguns recursos avançados que não abordaremos em profundidade, como:

- Interfaces e Classes Abstratas
- *Traits*
- *Namespaces*

Recursos Avançados

As ***Traits*** facilitam a reutilização de código em múltiplas classes sem a necessidade de herança múltipla.



Recursos Avançados

INSTITUTO FEDERAL
São Paulo

As ***Traits*** são uma maneira de reuso de código.

A intenção de uma Trait é **reduzir limitações de herança simples** permitindo que um desenvolvedor **reutilize** livremente conjuntos de **métodos** em **várias classes independentes** habitando em diferentes hierarquias de classe. - FONTE

Recursos Avançados

INSTITUTO FEDERAL
São Paulo

O problema: O PHP suporta apenas **herança única**: uma **classe filha pode herdar apenas de um único pai**.

Então, e se uma classe precisa herdar múltiplos comportamentos é preciso usar ***Traits*** para resolver o problema.

Recursos Avançados

INSTITUTO FEDERAL
São Paulo

Os ***namespaces***, em uma definição mais ampla, os *namespaces* (na tradução literal, "espaços de nomes") são uma **forma de encapsular itens**.

Recursos Avançados

INSTITUTO FEDERAL
São Paulo

De acordo com a documentação oficial do PHP os ***namespaces*** são projetados para **resolver dois problemas** que os autores de bibliotecas e aplicações encontram ao criar elementos de código reutilizáveis, como classes ou funções:

Recursos Avançados

INSTITUTO FEDERAL
São Paulo

1) **Colisões de nomes** entre o código criado e classes/funções/constantes internas do PHP ou classes/funções/constantes de terceiros.

2) **Capacidade de criar apelidos** (ou encurtar) Nomes_Extralongos projetados para aliviar o primeiro problema, melhorando a legibilidade do código-fonte.

Recursos Avançados

INSTITUTO FEDERAL
São Paulo

De acordo com W3Schools os namespaces:

1) permitem uma **melhor organização** ao agrupar classes que trabalham juntas para executar uma tarefa;

2) permitem que **o mesmo nome seja usado para mais de uma classe**.

Recursos Avançados

INSTITUTO FEDERAL
São Paulo

Por exemplo, você pode ter um conjunto de classes que descrevem uma tabela HTML, como ***Table***, ***Row*** e ***Cell***, enquanto também tem outro conjunto de classes para descrever móveis, como ***Table***, ***Chair*** e ***Bed***.

Recursos Avançados

INSTITUTO FEDERAL
São Paulo

Namespaces podem ser usados para organizar as classes em dois grupos diferentes, enquanto também **impedem** que as duas classes "*Table*" (HTML) e "*Table*" (Móvel) sejam misturadas.

Recursos Avançados

INSTITUTO FEDERAL
São Paulo

Por fim vale citar que o ***PHP Data Objects (PDO)***, o Laravel e o ***Eloquent*** (ORM do Laravel) serão abordados nas próximas aulas.

INSTITUTO FEDERAL
São Paulo

Referências

Referências

Leitura complementar e referências:

https://www.w3schools.com/php/php_oop_what_is.asp

https://www.w3schools.com/php/php_oop_classes_objects.asp

Referências

Leitura complementar e referências:

https://www.w3schools.com/php/php_oop_static_methods.asp

https://www.php.net/manual/pt_BR/language.oop5.basic.php

Referências

Leitura complementar:

https://programadoresdepre.com.br/o-que-e-programacao-orientada-a-objetos-poo/

https://www.bosontreinamentos.com.br/analise-de-sistemas/conceitos-de-programacao-orientada-a-objetos

Referências

Leitura complementar:

https://www.devmedia.com.br/principais-conceitos-da-programacao-orientada-a-objetos/32285

https://www.devmedia.com.br/orientacao-a-objetos-e-pdo-no-php/32644

Referências

Leitura complementar:

https://www.php.net/manual/pt_BR/language.oop5.traits.php

https://www.w3schools.com/php/php_oop_traits.asp

Referências

Leitura complementar:

https://www.php.net/manual/pt_BR/language.namespaces.rationale.php

https://www.w3schools.com/php/php_namespaces.asp

Referências

INSTITUTO FEDERAL
São Paulo

Conteúdo complementar (em vídeo):

https://youtu.be/QY0Kdg83orY

Programação Orientada a Objetos (POO) // Dicionário do Programador

209 mil visualizações • há 4 anos

Código Fonte TV

Ele é uma ideia antiga, que tomou vida graças a grande ajuda da HostGator Brasil. O tema da vez é: Programação **Orientada a** ...

Entenda Programação Orientada a Objetos | O que é Herança? | O que é Polimorfismo? | Como... 5 capítulos

INSTITUTO FEDERAL
São Paulo

Dúvidas?

Perguntas?

Sugestões?